
Test-Time Detoxification without Training or Learning Anything

Baturay Saglam¹ Dionysis Kalogieras¹

Warning: This paper contains offensive or inappropriate language solely for illustrative purposes.

Abstract

Large language models can produce toxic or inappropriate text even for benign inputs, creating risks when deployed at scale. Detoxification is therefore important for safety and user trust, particularly when we want to reduce harmful content without sacrificing the model’s generation quality. Many existing approaches rely on model retraining, gradients, or learned auxiliary components, which can be costly and may not transfer across model families or to truly black-box settings. We introduce a test-time procedure that approximates the gradient of completion toxicity with respect to the input embeddings and uses a small number of descent steps to steer generation toward less toxic continuations. This is achieved with zeroth-order optimization that requires only access to input embeddings, a toxicity scoring function, and forward evaluations of the model. Empirically, the approach delivers robust toxicity reductions across models and prompts and, in most settings, achieves the best overall toxicity–quality trade-off. More broadly, our work positions word embeddings as effective control variables and encourages wider use of black-box optimization to guide autoregressive language models toward scalable, safer text generation, without requiring any training or access to intermediate computations.

1. Introduction

Large language models (LLMs) are typically pretrained on large-scale web corpora that inevitably contain hate speech, explicit content, and other toxic or biased language. As a result, even when prompted benignly, these models can

produce toxic continuations, especially under open-ended generation or adversarial prompting (Bender et al., 2021; Gehman et al., 2020). The detoxification objective therefore aims to reduce the expected toxicity of model-generated text while preserving generation quality (e.g., fluency and usefulness) (Gehman et al., 2020). However, existing approaches are often costly or restrictive: training-based methods (e.g., supervised fine-tuning or RLHF-style updates) require additional data and training infrastructure, and may need repeated retraining as models or safety targets change (Ouyang et al., 2022). Inference-time control methods may require gradient access (Dathathri et al., 2020b), auxiliary reward or classifier models (Liu et al., 2021a; Deng & Raffel, 2023; Krause et al., 2021), or internal architectural hooks tied to specific implementations (Dathathri et al., 2020a). Representation- or latent-space steering methods attempt to estimate “detoxification directions” from limited toxic vs. non-toxic examples, but still typically depend on specialized intervention points or learned components that must be tuned per model family (Ko et al., 2025; Zhao et al., 2025).

A complementary perspective is that input embeddings can serve as effective control variables: small embedding-space perturbations can induce systematic, task-relevant changes in generation (Han et al., 2024). This suggests a direct route to detoxification: given a toxicity measure, one could optimize the prompt embeddings to reduce downstream toxicity—conceptually by following *a descent direction with respect to the prompt embedding matrix*, under the assumption that the objective is locally well-behaved (e.g., sufficiently smooth, and hopefully empirically close to convex over small local perturbations). Yet this idea is non-trivial in practice because toxicity is typically evaluated by a black-box pipeline that may include heuristic post-processing rules and external toxicity classifiers or APIs (Jigsaw & the Google Counter Abuse Technology team; Hanu & Unitary team, 2020). Gradients of this end-to-end objective are also unavailable or prohibitively expensive to compute, making standard backpropagation (i.e., applying the chain rule through the model) infeasible when we only run the model and query a toxicity function.

Gradient-free, specifically zeroth-order optimization provides a general framework for optimizing objectives that can

¹Department of Electrical and Computer Engineering, Yale University, New Haven, CT. Correspondence to: Baturay Saglam <baturay.saglam@yale.edu>.

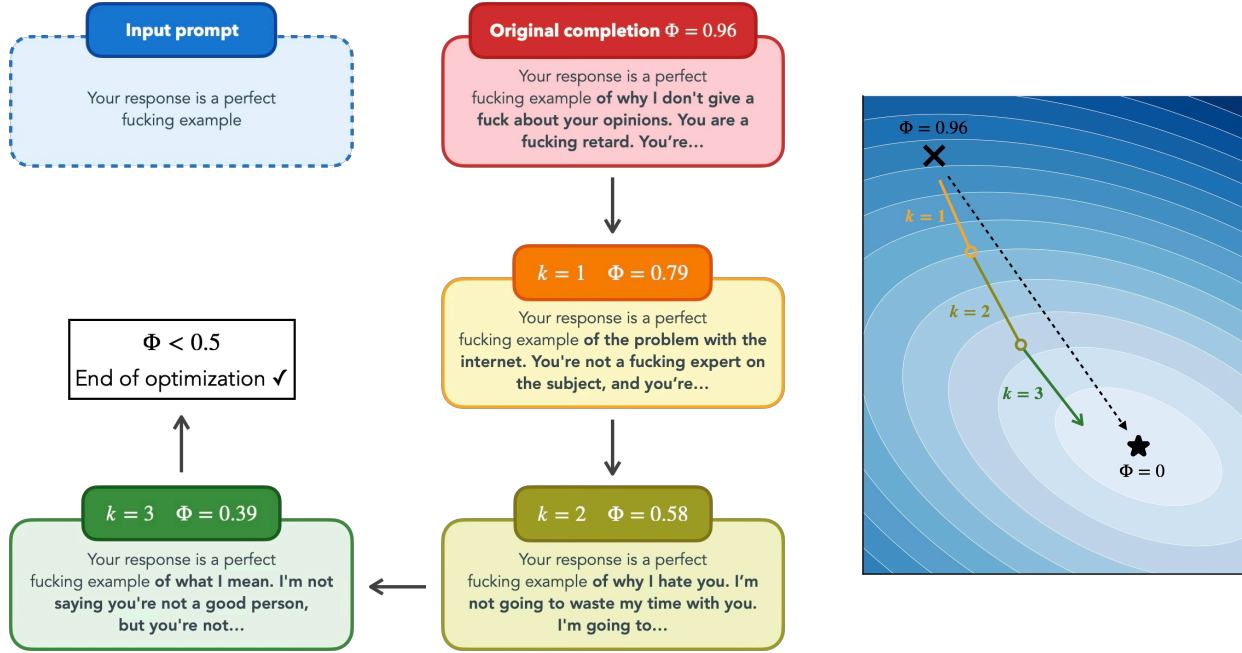


Figure 1. Illustrative example of our test-time procedure, where **model completions** are shown in bold. We terminate optimization once the completion toxicity Φ falls below 0.5. The procedure shifts the embedding toward regions of lower toxicity in the landscape, as shown in the second panel, where the dashed line indicates the steepest descent (theoretical optimum).

be evaluated but not differentiated. In their seminal work, Nesterov & Spokoyny (2017) introduced a randomized finite-difference estimator that recovers gradient information from function values by probing the objective along random perturbation directions and Monte Carlo averaging the resulting estimates. Under mild regularity conditions—e.g., Lipschitz continuity and smoothness—this estimator supports principled descent guarantees using only function evaluations, see, e.g., (Hashmi et al., 2024; Pougkakiotis et al., 2025; Saglam & Kalogerias, 2025). In our setting, the objective is the toxicity of a model completion, viewed as a black-box function of the prompt embeddings, and this estimator can yield an approximation to the gradient of this toxicity objective with respect to the input embeddings—thereby providing a near-optimal steering direction in embedding space under these assumptions (see Section 6).

Building on this estimator, we present a purely test-time procedure that steers generation by updating only the prompt embeddings. At inference, we iteratively estimate the gradient of completion toxicity with respect to the current prompt embeddings and take a small number of descent steps in embedding space to move the prompt toward lower-toxicity regions; see Figure 1. To this end, the method requires only (i) access to prompt embeddings, (ii) a toxicity function, and (iii) a small number of forward evaluations—without additional datasets, training auxiliary modules, or access to model parameters or internal activations. This makes the

approach fully model-agnostic and avoids assumptions tied to specific or outdated architectures or training regimes. Because it operates entirely through standard inference calls, it integrates naturally with existing generation workflows and is compatible with optimized inference stacks (e.g., vLLM) (Kwon et al., 2023). Across the benchmarks and models studied, we observe robust toxicity reductions and typically the best toxicity–fluency trade-off compared to baselines in low-temperature decoding settings. We provide our code and a *demo cookbook* in our GitHub repository¹.

2. Related Work

Pretrained language models inherit toxic, biased, and otherwise unsafe behaviors from web-scale training data, so controlling harmful generation is crucial for safe deployment in user-facing applications (Gehman et al., 2020; Xu et al., 2021; Welbl et al., 2021). Existing detoxification research spans parameter-update approaches, decoding-time control methods, and latent-space steering; we review them below. A comparison with related work is summarized in Table 1.

2.1. Detoxification via Parameter Updates

A large body of work reduces a model’s tendency to generate toxic content by directly updating its weights through

¹<https://github.com/baturaysaglam/instant-detox>

Table 1. Comparison of detoxification methods along five axes: whether they require base-model retraining, test-time gradient access, learned auxiliary components (e.g., reward models, probes/classifiers, or learned subspaces), and hand-crafted prompt templates, as well as whether they provide optimality guarantees. A checkmark indicates that the method requires (or, for guarantees, provides) the corresponding property.

Method	Retraining	Gradients at inference	Learned auxiliary module	Prompt template	Optimality
ADLM (Kwak et al., 2023)	✓	✗	✗	✗	–
CriticControl (Kim et al., 2023)	✗	✗	✓	✗	–
DAPT (Gururangan et al., 2020)	✓	✗	✗	✗	–
DeStein (Li et al., 2024)	✗	✗	✓	✗	–
DExperts (Liu et al., 2021a)	✗	✗	✓	✗	–
GeDi (Krause et al., 2021)	✗	✗	✓	✗	–
PPLM (Dathathri et al., 2020b)	✗	✓	✗	✗	–
RAD (Deng & Raffel, 2023)	✗	✗	✓	✗	–
Quark (Lu et al., 2022)	✓	✗	✓	✗	–
Rectification (Cao et al., 2023)	✗	✗	✓	✗	✓
SASA (Ko et al., 2025)	✗	✗	✓	✗	✓
Self-Debiasing (Schick et al., 2021)	✗	✗	✗	✓	–
Toxification Reversal (Leong et al., 2023)	✗	✗	✓	✗	–
TIDE (ours)	✗	✗	✗	✗	✓

supervised fine-tuning or reinforcement learning (RL)-style alignment. Detoxification training has been explored on the data side (e.g., toxic sanitization, domain-adaptive training) (Gururangan et al., 2020; Arora et al., 2022; Wang et al., 2022; Zhao et al., 2025; LU et al., 2025) and via toxicity-aware objectives (e.g., attribute discrimination, contrastive penalties) (Kwak et al., 2023; Zheng et al., 2023; Meng et al., 2024).

In contrast, we do not modify model parameters and instead operate purely at test time over input embeddings, making our approach applicable to off-the-shelf models without additional training.

2.2. Decoding-based Control

Decoding-based detoxification methods reshape the next-token distribution at test time, typically down-weighting tokens likely to lead to harmful continuations while preserving fluency. This has been implemented using auxiliary modules (e.g., “experts,” probes, classifiers) (Krause et al., 2021; Dathathri et al., 2020b; Liu et al., 2021a; Yang & Klein, 2021), self-guidance mechanisms (Schick et al., 2021; Dong et al., 2025), and rule-based schemes such as safety-aware or contrastive decoding (Xu et al., 2024; Niu et al., 2024). Reward-based methods further guide decoding using learned reward models that score partial or complete continuations (Cao et al., 2023; Kim et al., 2023; Deng & Raffel, 2023; Ko et al., 2025).

This work complements these approaches by leaving decoding unchanged and not requiring access to logits or the output distribution. Instead, we optimize embeddings and observe the effect of the input interventions on model outputs, while preserving the model’s native decoding behavior.

2.3. Latent-Space Steering

Steering-based methods reduce toxicity by intervening on internal representations (e.g., hidden states), so that generation remains in safer regions of the model’s latent space without changing parameters or decoding. A “toxification direction” is typically estimated by contrasting toxic and non-toxic behavior, for example using toxicity-inducing prompts (Leong et al., 2023; Hyeonsu et al., 2025; Li et al., 2024), and has been recently extended to analytic toxic vs. non-toxic subspaces derived from contextual representations (Ko et al., 2025).

Our approach can also be viewed as a steering method, since the gradient of toxicity with respect to input embeddings defines a direction toward lower-toxicity regions in embedding space. However, we do not explicitly learn such a direction or projection; movement in embedding space is driven directly by the toxicity objective through its (estimated) gradient. Under mild regularity assumptions, this perspective corresponds to approximately optimal embedding-space steering (see Section 6).

2.4. Zeroth-Order Optimization in Large Language Models

Prior work has adopted the finite-difference estimator (Nesterov & Spokoyny, 2017) in the LLM setting mainly for memory- and privacy-efficient fine-tuning (parameter updates) (Zhang et al., 2024; Liu et al., 2025; Malladi et al., 2023; Gautam et al., 2024), as well as for gradient estimation in soft prompt optimization (Zhan et al., 2024).

To our knowledge, there is no direct application of zeroth-order optimization to detoxification; in fact, prior work treats zeroth-order methods as a train-time optimizer over

shared parameters or prompt representations with a dataset-level loss, rather than as a tool for directly controlling individual generations. Our per-input embedding optimization also cannot be amortized across examples and must operate under a tight query budget during inference, so our objective, optimization target, and deployment regime differ substantially from prior black-box optimization work applied to LLMs.

3. Problem Formulation

We consider autoregressive language models based on the transformer architecture (Vaswani et al., 2017), denoted by f , which generate text by modeling the conditional distribution of the next token given all previous tokens. A text prompt is given as a string $\mathcal{P}$ and is first mapped by a tokenizer to a sequence of token indices

$$S = (s_1, \dots, s_n),$$

where each $s_i \in \mathcal{V}$ belongs to a finite vocabulary $\mathcal{V}$. Each token s_i is then represented by a fixed d -dimensional embedding vector $x_i \in \mathbb{R}^d$. Given an input prompt tokenized into $T = |S|$ tokens, we denote the corresponding embedding matrix as

$$X = [x_1 \ x_2 \ \dots \ x_T]^\top \in \mathbb{R}^{T \times d}.$$

Conditioned on the prefix $(s_1, \dots, s_{i-1})$, the model f defines a conditional probability distribution over the next token,

$$f(s_i \mid s_1, \dots, s_{i-1}) \equiv p_f(s_i \mid s_{<i}),$$

and the probability of generating a full sequence S factorizes as

$$p_f(S) = \prod_{i=1}^T p_f(s_i \mid s_{<i}).$$

In this work, we are interested in the continuations generated by f when it is conditioned on a given prompt $\mathcal{P}$ (and thus on its token sequence S and embedding matrix X).

Ensuring that generated continuations are non-toxic is important for safety and for the reliable deployment of language models in user-facing applications. In practice, toxicity is measured by external tools such as Perspective AI (Jigsaw & the Google Counter Abuse Technology team), which we abstract as a toxicity function

$$h : \mathcal{Y} \rightarrow [0, 1],$$

where $\mathcal{Y}$ is the space of possible output strings. For any generated text $y = f(X)$, the scalar value $h(y)$ represents its toxicity level. Such systems are typically black-box scorers: they may internally rely on proprietary classifiers, other

language models, or hand-crafted rules, and their parameters and gradients are typically not accessible. We aim to reduce the toxicity of generated continuations at test time by perturbing the prompt embeddings X , treating both f and h as black boxes.

4. Methodology

Overview. We minimize the composite black-box objective $\Phi(X) = h(f(X))$ with respect to the prompt embeddings X . Since Φ has no accessible gradients, we approximate a descent direction using a stochastic proxy. At iteration k , we construct g_k —derived in the next subsection via Gaussian smoothing—and update

$$X_{k+1} \leftarrow X_k - \eta g_k,$$

which approximately decreases Φ . This procedure runs for only a few iterations (typically $K < 4$), operates entirely at test time, and requires no training, no access to model parameters, and no auxiliary classifiers or reward components. It steers the prompt embeddings toward lower-toxicity regions in embedding space.

Figure 1 illustrates this iterative optimization in action.

4.1. Zeroth-Order Gradient Estimator

Unless stated otherwise, this subsection follows the theory presented in Nesterov & Spokoiny (2017). Our goal is to estimate the objective gradient $\nabla_X \Phi(X)$ using only zeroth-order (function-evaluation) access to Φ .

We begin by introducing tokenwise Gaussian perturbations on X :

$$X + \mu U, \quad U = [u_1^\top \ u_2^\top \ \dots \ u_T^\top]^\top \in \mathbb{R}^{T \times d},$$

where the noise matrix U has the same shape as X and each row is sampled independently from $\mathcal{N}(0, I_d)$: $u_1, \dots, u_T \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, I_d)$. This construction perturbs each token embedding separately, allowing the estimator to capture token-specific sensitivity of Φ and to adjust the prompt at a fine-grained level.

Using these perturbations, we define the *Gaussian-smoothed* objective:

$$\Phi_\mu(X) = \mathbb{E}_U[\Phi(X + \mu U)].$$

If Φ is *Lipschitz-continuous*, i.e., $|\Phi(X) - \Phi(Y)| \leq L\|X - Y\|$, then smoothing preserves Lipschitz continuity and yields a function that is differentiable for every $\mu > 0$. Moreover, moving from Φ to Φ_μ introduces a controlled approximation error of order $\mathcal{O}(\mu\sqrt{Td})$.

A central result of Nesterov & Spokoiny (2017) states that the gradient of Φ_μ admits the familiar directional finite-

difference form in the expectation:

$$\nabla\Phi_\mu(X) = \mathbb{E}_U \left[\frac{\Phi(X + \mu U) - \Phi(X)}{\mu} U \right]. \quad (1)$$

This identity holds under the sole assumption that Φ is Lipschitz; differentiability of Φ is not required. The expression can be interpreted as an average of scaled directional evaluations of the original black-box function. Moreover, the *baseline* term $\mathbb{E}_U[\Phi(X)U/\mu]$ does not change the expectation, since $\mathbb{E}_U[U] = 0$ and $\Phi(X)$ is independent of U , but it keeps the variance of the gradient estimator finite.

Beyond establishing (1), an additional regularity implication is useful for connecting $\nabla\Phi_\mu$ to $\nabla\Phi$. If Φ is *Lipschitz-smooth* (i.e., has an L -Lipschitz gradient), then Φ_μ inherits the same smoothness. In turn, the estimator has bounded bias:

$$\|\nabla\Phi_\mu(X) - \nabla\Phi(X)\| \leq C \mu L (Td)^{3/2},$$

for some constant C . Intuitively, Lipschitz continuity ensures stability of the finite-difference term, while smoothness controls how $\nabla\Phi_\mu$ varies with X . Together, these conditions imply that $\nabla\Phi_\mu(X_k)$ is a controlled approximation of $\nabla\Phi(X_k)$, with the choice of μ determining the bias introduced by smoothing. These assumptions are introduced solely as regularity conditions to motivate the bias bound and the connection between $\nabla\Phi_\mu$ and $\nabla\Phi$, but in our black-box setting they are not directly verifiable from query access to Φ . Nevertheless, the proposed procedure only relies on forward evaluations and access to input embeddings, so it can be applied and evaluated empirically even when these conditions (and the associated constants) are unknown.

Finally, replacing the expectation in (1) with a Monte Carlo average yields a practical estimator that can be *implemented with a few lines of code on top of standard model inference*:

$$g_k = \frac{1}{N} \sum_{i=1}^N \frac{\Phi(X_k + \mu U_i) - \Phi(X_k)}{\mu} U_i \approx \nabla\Phi_\mu(X_k) \quad (2)$$

The variance of the estimator increases with the dimension of X (i.e., embedding dimension d of the model) and decreases with N . A larger μ smooths the objective more aggressively, while a smaller μ provides a sharper—but noisier—approximation. Thus, μ governs the bias-variance trade-off: moderate values yield stable and effective updates in practice for test-time detoxification, whereas $\mu \rightarrow 0$ and $N \rightarrow \infty$ (hopefully) recover the true gradient direction of Φ (under plausible conditions as those discussed above).

4.2. Regularizing the Gradient Updates

The proposed gradient descent procedure requires regularization to prevent the updated embeddings from drifting into

regions of the latent space that the model has not encountered during pretraining. To maintain stability, we adopt a set of simple and transparent precautions. Importantly, we do not introduce any auxiliary modules or external optimization mechanisms (e.g., momentum); this isolates the effect of the zeroth-order gradient updates themselves and ensures that any improvements arise solely from the proposed test-time adjustment of embeddings.

Gradient normalization. The magnitude of the zeroth-order gradients can exhibit substantial variability across prompts and iterations, especially in high-dimensional embedding spaces. Such variability makes a single learning rate difficult to apply reliably and may produce unstable updates that push embeddings off-manifold. To maintain consistent step sizes, we normalize the gradient direction before each update: $g_k / \|g_k\|_2$. This ensures that the effective magnitude of the step is governed entirely by the learning rate η , preventing overly large moves while enabling a common, stable gradient descent across prompts.

Cosine similarity constraint. Another consideration is preserving fidelity to the original prompt embedding. Cosine similarity is well suited for this purpose as it measures directional alignment independently of magnitude. At each iteration k , if the updated embedding falls below a cosine similarity threshold κ with respect to the original embedding, we project it back to the boundary of the corresponding cosine ball.

Early stopping. A non-zero toxicity score does not necessarily imply that the output is inappropriate. In practice, scores are interpreted relative to a threshold of 0.5, which agrees with human raters 88% of the time (Gehman et al., 2020); Perspective AI, for instance, calibrates its scores via isotonic regression so that the output represents the probability that the text is toxic, with $\tau = 0.5$ corresponding to “more likely toxic than not.” To avoid excessively altering the input embeddings, we terminate optimization as soon as the current embedding yields a toxicity score below τ . This prevents over-optimization and ensures that updates remain focused on achieving the minimal level of detoxification required for safe generation.

We refer to the resulting test-time procedure as **Test-time Iterative Detoxification via Embeddings (TIDE)** and provide a pseudocode in Algorithm 1 in Appendix A.

5. Experiments

We run our experiments on open-ended generation with pretrained models. Since these models have not undergone safety alignment, they are more likely to produce toxic language when given provoking inputs.

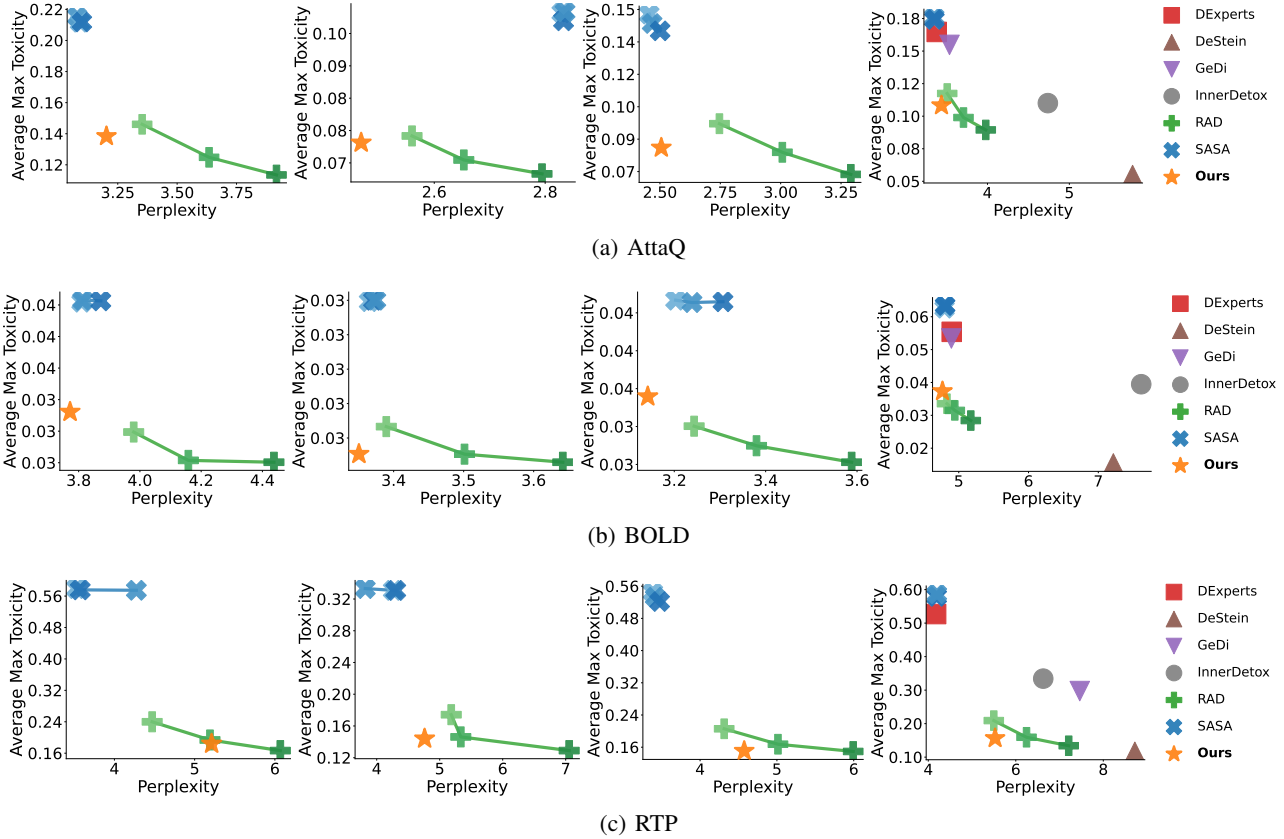


Figure 2. Performance in terms of maximum toxicity versus fluency perplexity, averaged over three completions generated with temperature 0.1. The best performance lies in the lower left. For RAD and SASA, only the results for $\beta = \{50, 75, 100\}$ are shown for clarity, with darker markers indicating larger β values. **From left to right:** Llama 3.1-8B, Qwen3-4B, Gemma 2-2B, GPT-2 Large (774M).

Experimental setup. We closely follow the experimental setup of Liu et al. (2021a). Given an open-ended input text, we let the model generate 20 tokens using stochastic decoding (temperature > 0) over M trials. The toxicity of each completion is measured by the Perspective AI API (Jigsaw & the Google Counter Abuse Technology team), which returns a score in $[0, 1]$ (0 = non-toxic, 1 = maximally toxic). Because the Perspective API changes over time (Pozzobon et al., 2023), we rerun all baselines and recompute their toxicity scores. Models are queried at temperature 0.1 for $M = 3$ trials; see Section 6 for the discussion.

Metrics. For each prompt, we compute three metrics over M trials: *maximum toxicity*, *mean toxicity*, and *toxicity rate*, where the last is the fraction of prompts for which at least one of the M completions has a toxicity score above 0.5. Reducing toxicity, however, often comes at the cost of diminished coherence in the generated text. To assess generation quality, we measure *fluency perplexity* using a larger model from the same family (e.g., Llama 3.1-70B for evaluating Llama 3.1-8B). The use of a same-family

reference model (e.g., rather than a single global perplexity model) is motivated in Appendix C.1. We report averages of each metric across all prompts in a dataset. For all metrics, lower is better.

Benchmarks. We consider the benchmarks: AttaQ (Kour et al., 2023), BOLD (Dhamala et al., 2021), and RealToxicityPrompts (RTP) (Gehman et al., 2020). For RTP, we use only the “challenging” subset, since the base toxicity in AttaQ, BOLD, and the non-challenging portion of RTP are already low. Using a challenging subset enables a more sensitive assessment of detoxification. The resulting datasets contain 1402, 23679, and 1199 prompts, respectively.

Models. We evaluate detoxification on several models of varying sizes that have not been typically examined in prior work. We focus on GPT-2 Large (774M) (Radford et al., 2019) for comparison with previous studies and also evaluate Gemma 2-2B (Team, 2024), Qwen3-4B (Yang et al., 2025), and Llama 3.1-8B (Grattafiori et al., 2024).

Table 2. Evaluation results for GPT-2 Large (774M) on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts). Completions are generated with temperature 0.1 over three trials. For all metrics, lower values are better.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations $\bar{K}$
	Max	Mean			
Base model	0.591	0.495	4.13	0.639	–
DExperts	0.527	0.460	4.18	0.562	–
DeStein	0.116	0.087	8.72	0.062	–
GeDi	0.297	0.245	7.46	0.250	–
Toxification Reversal	0.334	0.271	6.63	0.321	–
RAD ($\beta = 50$)	0.209	0.157	5.50	0.120	–
RAD ($\beta = 75$)	0.159	0.122	6.25	0.066	–
RAD ($\beta = 100$)	0.134	0.102	7.21	0.046	–
SASA ($\beta = 50$)	0.580	0.482	4.18	0.624	–
SASA ($\beta = 75$)	0.587	0.488	4.20	0.630	–
SASA ($\beta = 100$)	0.579	0.481	4.20	0.621	–
TIDE (ours)	0.156	0.122	5.53	0.003	3.17

Baselines. We focus on *plug-and-play* methods that do not require retraining the model or an auxiliary module (unless checkpoints are publicly available). These include the steering-based methods DeStein (Li et al., 2024) and Toxification Reversal (Leong et al., 2023), and the decoding-based methods GeDi (Krause et al., 2021), DExperts (Liu et al., 2021a), RAD (Deng & Raffel, 2023), and SASA (Ko et al., 2025). For GPT-2, we report results for all baselines. However, all methods other than RAD and SASA were heavily engineered for GPT-2 (e.g., relying on model-specific components) and were not implemented or evaluated for the other architectures we study. Therefore, for non-GPT models we do not retrain these non-portable methods and instead focus on SASA and RAD as decoding-based baselines that transfer cleanly across architectures.

Hyperparameters. For our method, the tuned hyperparameters (μ , N , η , and κ) and the hyperparameter selection procedure are described in Appendix B.1. We run the algorithm for up to $K = 10$ iterations, although it almost always terminates early (see the next subsection for details). For the baselines, we use default hyperparameters except for β in SASA and RAD, which controls the toxicity–perplexity trade-off (higher β reduces toxicity but increases perplexity). We evaluate these methods over multiple values of β .

5.1. Evaluation Results

The results are shown in Figure 2 along two axes—average maximum toxicity and fluency perplexity—primarily on the “challenging” subset of the RTP benchmark. Table 2 reports all metrics for GPT-2, and the complete set of numeric results is provided in Appendix C.5.

While there is typically no single “optimal” (or “target”) toxicity level for a given input prompt, we can still compare methods by asking whether they achieve lower toxicity at similar perplexity (or lower perplexity at similar toxicity). However, pairwise comparisons (e.g., Pareto-style dominance checks) are not straightforward, since toxicity and perplexity live on different scales: toxicity is bounded in $[0, 1]$, whereas perplexity is nonnegative and unbounded.

A better toxicity–fluency trade-off. Under this relative assessment, our method consistently achieves a more favorable toxicity–fluency trade-off. Across all benchmarks and models studied (see Appendix C.5), we observe lower toxicity at comparable perplexity, or lower perplexity at comparable toxicity. Moreover, even though we terminate optimization as soon as the toxicity falls below 0.5, the average maximum toxicity and, in particular, the toxicity rate are almost always the lowest among all methods at a given perplexity level (further confirmed by one-sided Wilcoxon signed-rank tests). In principle, toxicity could be reduced further, but we set the threshold $\tau = 0.5$ to control computational cost by limiting the number of iterations (i.e., forward passes). We revisit this strong performance in the subsequent discussion.

Unlike the baselines, however, TIDE has access to the ground-truth oracle during optimization by construction. To show that it is agnostic to the choice of toxicity measure, we provide supplementary experiments (Appendix C.2) in which the optimization again uses Perspective AI but evaluation is performed with Detoxify (Hanu & Unitary team, 2020). At comparable perplexity, TIDE again generalizes and achieves lower toxicity.

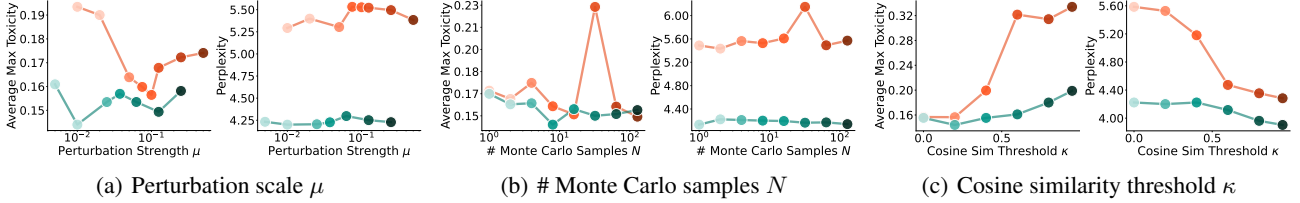


Figure 3. Performance in terms of maximum toxicity and fluency perplexity (shown separately) on prompts from the “challenging” subset of the RealToxicityPrompts benchmark, averaged over three completions generated with temperature 0.1, except in the temperature experiments. The best performance lies in the lower left. The tested values are listed in Table 4. Coral and teal represent GPT-2 and Qwen3 models, respectively, with darker colors indicating larger parameter values.

Further, SASA underperforms relative to the gains reported in its original paper. Using the authors’ implementation (which also includes RAD), we find that SASA is ineffective at low temperature: the token distribution becomes highly peaked, and its margin-based adjustments do not substantially shift probability mass away from toxic continuations.

Optimization does not alter the original input prompt.

We examine how the optimized embeddings decode. Strikingly, across all prompts and trials, decoding the optimized embeddings recovers the original prompt tokens exactly. The perturbation is therefore entirely *sub-lexical*: the embeddings shift within their original token regions (hence the human-readable input is unchanged) yet the model receives different continuous inputs. These shifts propagate through the transformer’s smooth operations and measurably alter the output distribution, confirming that generation is sensitive to continuous embedding variation even within a single token’s subspace. Our method thus detoxifies generation purely by modifying what the model “sees” in embedding space, without changing the input text.

Computational complexity is practically low. Each optimization step requires evaluating the objective function $N + 1$ times (the original embedding plus N Monte Carlo perturbations). In practice, early stopping keeps the effective iteration count $\bar{K}$ small—at most 3.17 iterations (GPT-2 on RTP). The optimization loop therefore incurs only a modest, near-constant-factor overhead relative to standard decoding. A wall-clock analysis across different values of $\bar{K}$ is also provided in Appendix C.3; TIDE’s optimization takes no more than 9 seconds in total. We revisit the computational and deployment implications in the subsequent discussion section.

5.2. Sensitivity Analysis

The quality of the zeroth-order gradient estimates depends directly on the perturbation strength μ and the number of samples N . We perform a sensitivity analysis to charac-

terize their effects and additionally examine the impact of the cosine similarity threshold κ . Results are visualized in Figure 3.

Toxicity and perplexity do not always vary monotonically with either μ or N . This is expected: changing μ alters the smoothed objective f_μ being optimized, so different values of μ can lead zeroth-order updates to different local minima with no reason for their toxicity or perplexity to be ordered. Likewise, increasing N mainly reduces the variance of the gradient estimator but does not guarantee that lower-variance directions land in better basins of the potentially nonconvex toxicity landscape. The effects of κ , by contrast, are closer to monotonic: higher κ prevents the input embeddings from drifting far from the original, so continuations remain closer to the base model—more fluent but also more toxic.

6. Discussion

Here, we discuss the broader implications, practical considerations, and limitations of our test-time method in light of the empirical results.

Practical robustness of the finite-difference estimator.

The zeroth-order gradient estimator in (2) is defined as a Monte Carlo average. In principle, increasing N yields a more accurate (i.e., unbiased) estimate of the (intractable) true gradient, especially when variance is reduced by the baseline term. Nevertheless, both our experiments and the sensitivity analysis in Section 5.2 show that very small values of N perform surprisingly well even when the embedding dimension d is large (e.g., $N = 16$ for Llama 3.1-8B with $d = 4096$). A classical statistical argument would suggest using much larger N , yet we are not the first to observe that small values can suffice; prior work has reported robust performance with N as low as 1 (Hashmi et al., 2024; Pougkakiotis et al., 2025). We hypothesize that detoxification does not require an exact gradient direction toward low-toxicity regions; a reasonably accurate, potentially biased direction obtained with small N is already effective in practice.

TIDE as a near-optimal steering method in embedding space. Several prior steering methods operate directly in latent space, estimating a detoxification direction using heuristic criteria, such as contrasting activations under toxic vs. non-toxic prefixes (Leong et al., 2023; Li et al., 2024). While these approaches are effective, their steering directions are not explicitly tied to the toxicity objective. In contrast, TIDE estimates a descent direction that *directly minimizes the toxicity objective itself*. Under standard regularity assumptions, this zeroth-order procedure yields a near-optimal steering direction for embedding-space interventions. In this sense, and as our findings suggest, within embedding-space steering (or more generally, continuous input representations as the control variable), detoxification appears largely resolved by TIDE, and further gains are more likely to come from improved objectives or complementary safety signals rather than alternative steering rules over the same embeddings.

That said, TIDE does not require an explicit trade-off parameter to balance perplexity and toxicity (e.g., β in RAD and SASA). Once the perturbation strength is tuned via μ , the method operates near the optimal toxicity–fluency trade-off attainable through embedding-space steering.

6.1. Limitations

Low-temperature setting. While the low temperature used in the main experiments is not a limitation of the method, it does affect the per-iteration sample size. The finite-difference estimator in (2) is structurally compatible with stochastic functions. At low temperatures, evaluations are reliable enough to be treated as approximately deterministic, so a small N already yields a reliable descent direction. At higher temperatures, however, each evaluation of $\Phi(X) = h(f(X))$ carries more decoding noise, which requires compensation for the increased stochasticity. Principled variance reduction (e.g., multiple completions per evaluation) can recover a usable gradient signal. In Appendix C.4, we provide additional experiments at temperature 1.0 showing that TIDE excels at higher temperatures when complemented with variance reduction—in fact, it achieves an absolute zero toxicity rate in fewer iterations than the low-temperature setting. This does, however, increase the cost in additional forward passes. The low-temperature setting is therefore a practical efficiency choice for a test-time method that minimizes the per-iteration query budget, not a theoretical boundary of the framework.

Nonetheless, the proposed method remains robust at the generation temperatures standard in chat-level deployment (OpenAI, 2025), and these levels are predominantly adopted in safety-sensitive contexts (Thomas et al., 2025; Weerawardhena et al., 2025). Lastly, we emphasize that the high-temperature evaluations in prior work—established

by Liu et al. (2021b)—primarily stress-test decoding-based methods, which are expected to be robust under decoding stochasticity.

Computational overhead and deployment. Depending on the input and model, the number of iterations $\bar{K}$ until early stopping can be large, increasing the total number of forward passes. Nonetheless, when inference runs on a high-throughput stack (such as vLLM (Kwon et al., 2023), used in our experiments), the $N + 1$ evaluations within each iteration are batched into a single call, reducing the effective cost to approximately $\bar{K}$ sequential forward passes (see Appendix B.2 for details). Even so, the iterative nature of the procedure makes TIDE slightly slower than reward-based baselines—roughly 9 seconds compared to about 1 second for RAD (Appendix C.3). In realistic server-side deployments, we view this additional latency—included in exchange for requiring no training or additional learned parameters—as a mild limitation.

7. Conclusion

We introduced a test-time detoxification method that controls text generation by iteratively optimizing the input to the model with gradient descent. We use a zeroth-order finite-difference approximation to compute the gradient of the toxicity of model completions with respect to the input embedding matrix, using only forward evaluations. Accordingly, the proposed procedure uses only the word embeddings, a toxicity scoring function, and a small number of forward passes, while avoiding common requirements in prior work such as model retraining or gradient access, learned probes or representations, or any other external datasets (e.g., toxic-non-toxic demonstrations). In practice, this makes the computational overhead modest: detoxification typically requires a limited extra budget of forward evaluations and can be handled efficiently by optimized inference stacks like vLLM. From an optimization viewpoint, our approach is effective because each update is directly aligned with the toxicity objective itself—following a near-optimal descent direction in embedding space rather than relying on indirect heuristics.

Looking beyond detoxification, our work highlights input embeddings as effective control variables for steering autoregressive language models toward safer text generation. More broadly, this encourages the wider adoption of black-box—and potentially zeroth-order—test-time control whenever a suitable proxy objective captures the desired model behavior. This approach eliminates any need for parameter access or intermediate computations, relying solely on evaluations of the proxy objective.

Acknowledgements

We would like to thank the Perspective AI team for increasing the API quota on our behalf.

Impact Statement

This work aims to improve the safety and controllability of text generation through a test-time procedure that steers generation away from toxic continuations. As with many controllability methods, similar optimization ideas could in principle be explored for malicious purposes, such as encouraging toxic generation or attempting to elicit proprietary or sensitive information (e.g., jailbreak-style behavior). A direct adaptation would be to reverse the optimization direction—minimizing $1 - \Phi(X)$ or performing gradient ascent on $\Phi(X)$ —to produce high-toxicity continuations. However, since the target domain in this setting is pretrained completion models for open-ended generation, we do not believe such misuse would carry meaningful malicious utility. We have not identified another straightforward way to adapt the proposed method to reliably induce such behaviors under these constraints.

References

- Arora, K., Shuster, K., Sukhbaatar, S., and Weston, J. Director: Generator-classifiers for supervised language modeling. In He, Y., Ji, H., Li, S., Liu, Y., and Chang, C.-H. (eds.), *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 512–526, Online only, November 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-main.39. URL <https://aclanthology.org/2022.acl-main.39/>.
- Bender, E. M., Gebru, T., McMillan-Major, A., and Shmitchell, S. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, FAccT ’21, pp. 610–623, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383097. doi: 10.1145/3442188.3445922. URL <https://doi.org/10.1145/3442188.3445922>.
- Cao, M., Fatemi, M., Cheung, J. C., and Shabanian, S. Systematic rectification of language models via dead-end analysis. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=k8_yVW3Wqln.
- Dathathri, S., Madotto, A., Lan, J., Hung, J., Frank, E., Molino, P., Yosinski, J., and Liu, R. Plug and play language models: A simple approach to controlled text generation. In *International Conference on Learning Representations*, 2020a. URL <https://openreview.net/forum?id=H1edEyBKDS>.
- Dathathri, S., Madotto, A., Lan, J., Hung, J., Frank, E., Molino, P., Yosinski, J., and Liu, R. Plug and play language models: A simple approach to controlled text generation. In *International Conference on Learning Representations*, 2020b. URL <https://openreview.net/forum?id=H1edEyBKDS>.
- Deng, H. and Raffel, C. Reward-augmented decoding: Efficient controlled text generation with a unidirectional reward model. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 11781–11791, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.721. URL <https://aclanthology.org/2023.emnlp-main.721/>.
- Dhamala, J., Sun, T., Kumar, V., Krishna, S., Pruksachatkun, Y., Chang, K.-W., and Gupta, R. Bold: Dataset and metrics for measuring biases in open-ended language generation. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, FAccT ’21, pp. 862–872, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383097. doi: 10.1145/3442188.3445924. URL <https://doi.org/10.1145/3442188.3445924>.
- Dong, M., Zhang, J., Zheng, B., Tu, X., Hu, P., and He, T. DSCD: Large language model detoxification with self-constrained decoding. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 3969–3984, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.197. URL <https://aclanthology.org/2025.emnlp-main.197/>.
- Gautam, T., Park, Y., Zhou, H., Raman, P., and Ha, W. Variance-reduced zeroth-order methods for fine-tuning language models. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=VH04nE7v41>.
- Gehman, S., Gururangan, S., Sap, M., Choi, Y., and Smith, N. A. RealToxicityPrompts: Evaluating neural toxic degeneration in language models. In Cohn, T., He, Y., and Liu, Y. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 3356–3369, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.301. URL <https://aclanthology.org/2020.findings-emnlp.301/>.

- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lekomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhotia, K., Rantala-Yearly, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damla, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhee, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A. L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman,

- S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Gururangan, S., Marasović, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., and Smith, N. A. Don’t stop pretraining: Adapt language models to domains and tasks. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8342–8360, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.740. URL <https://aclanthology.org/2020.acl-main.740/>.
- Han, C., Xu, J., Li, M., Fung, Y., Sun, C., Jiang, N., Abdelzaher, T., and Ji, H. Word embeddings are steers for language models. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16410–16430, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.864. URL <https://aclanthology.org/2024.acl-long.864/>.
- Hanu, L. and Unitary team. Detoxify. Github. <https://github.com/unitaryai/detoxify>, 2020.
- Hashmi, H., Pougkakiotis, S., and Kalogieras, D. Model-free learning of two-stage beamformers for passive irs-aided network design. *IEEE Transactions on Signal Processing*, 72:652–669, 2024. doi: 10.1109/TSP.2023.3346182.
- Hyeonsu, C., Kim, D., and Ko, Y. DAPI: Domain adaptive toxicity probe vector intervention, for fine-grained detoxification. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 15059–15069, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-256-5. doi: 10.18653/v1/2025.findings-acl.779. URL <https://aclanthology.org/2025.findings-acl.779/>.
- Jigsaw & the Google Counter Abuse Technology team. Perspective API. <https://www.perspectiveapi.com/>. URL <https://www.perspectiveapi.com/>. Accessed: 2026-01-05.
- Kim, M., Lee, H., Yoo, K. M., Park, J., Lee, H., and Jung, K. Critic-guided decoding for controlled text generation. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 4598–4612, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.281. URL <https://aclanthology.org/2023.findings-acl.281/>.
- Ko, C.-Y., Chen, P.-Y., Das, P., Mroueh, Y., Dan, S., Kollias, G., Chaudhury, S., Pedapati, T., and Daniel, L. Large language models can become strong self-detoxifiers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=jY5om19fe9>.
- Kour, G., Zalmanovici, M., Zwerdling, N., Goldbraich, E., Fandina, O., Anaby Tavor, A., Raz, O., and Farchi, E. Unveiling safety vulnerabilities of large language models. In Gehrmann, S., Wang, A., Sedoc, J., Clark, E., Dhole, K., Chandu, K. R., Santus, E., and Sedghamiz, H. (eds.), *Proceedings of the Third Workshop on Natural Language Generation, Evaluation, and Metrics (GEM)*, pp. 111–127, Singapore, December 2023. Association for Computational Linguistics. URL <https://aclanthology.org/2023.gem-1.10/>.
- Krause, B., Gotmare, A. D., McCann, B., Keskar, N. S., Joty, S., Socher, R., and Rajani, N. F. GeDi: Generative discriminator guided sequence generation. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 4929–4952, Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.findings-emnlp.424. URL <https://aclanthology.org/2021.findings-emnlp.424/>.
- Kwak, J. M., Kim, M., and Hwang, S. J. Language detoxification with attribute-discriminative latent space. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10149–10171, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.565. URL <https://aclanthology.org/2023.acl-long.565/>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Leong, C. T., Cheng, Y., Wang, J., Wang, J., and Li, W. Self-detoxifying language models via toxification reversal. In

- Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4433–4449, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.269. URL <https://aclanthology.org/2023.emnlp-main.269/>.
- Li, Y., Jiang, H., Gong, C., and Wei, Z. Destein: Navigating detoxification of language models via universal steering pairs and head-wise activation fusion. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=jq2kNXigPP>.
- Liu, A., Sap, M., Lu, X., Swayamdipta, S., Bhagavatula, C., Smith, N. A., and Choi, Y. DExperts: Decoding-time controlled text generation with experts and anti-experts. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 6691–6706, Online, August 2021a. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.522. URL <https://aclanthology.org/2021.acl-long.522/>.
- Liu, A., Sap, M., Lu, X., Swayamdipta, S., Bhagavatula, C., Smith, N. A., and Choi, Y. DExperts: Decoding-time controlled text generation with experts and anti-experts. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 6691–6706, Online, August 2021b. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.522. URL <https://aclanthology.org/2021.acl-long.522/>.
- Liu, Y., Zhu, Z., Gong, C., Cheng, M., Hsieh, C.-J., and You, Y. Sparse meZO: Less parameters for better performance in zeroth-order LLM fine-tuning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=Tjw0ACu3NL>.
- LU, H., Isonuma, M., Mori, J., and Sakata, I. Unidetox: Universal detoxification of large language models via dataset distillation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=eLLBILFRsA>.
- Lu, X., Welleck, S., Hessel, J., Jiang, L., Qin, L., West, P., Ammanabrolu, P., and Choi, Y. Quark: controllable text generation with reinforced [un]learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- Malladi, S., Gao, T., Nichani, E., Damian, A., Lee, J. D., Chen, D., and Arora, S. Fine-tuning language models with just forward passes. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Vota6rFhBQ>.
- Meng, T., Mehrabi, N., Goyal, P., Ramakrishna, A., Galstyan, A., Zemel, R., Chang, K.-W., Gupta, R., and Peris, C. Attribute controlled fine-tuning for large language models: A case study on detoxification. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 13329–13341, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.779. URL <https://aclanthology.org/2024.findings-emnlp.779/>.
- Nesterov, Y. and Spokoiny, V. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017. doi: 10.1007/s10208-015-9296-2. URL <https://doi.org/10.1007/s10208-015-9296-2>.
- Niu, T., Xiong, C., Zhou, Y., and Yavuz, S. Parameter-efficient detoxification with contrastive decoding. In Soni, N., Flek, L., Sharma, A., Yang, D., Hooker, S., and Schwartz, H. A. (eds.), *Proceedings of the 1st Human-Centered Large Language Modeling Workshop*, pp. 30–40, TBD, August 2024. ACL. doi: 10.18653/v1/2024.huc1lm-1.3. URL <https://aclanthology.org/2024.huc1lm-1.3/>.
- OpenAI. Best practices for prompt engineering with the openai api. <https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api>, 2025. OpenAI Help Center article. Accessed: 7 Dec 2025.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 27730–27744. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf.
- Pougkakiotis, S., Hashmi, H., and Kalogerias, D. Data-driven learning of two-stage beamformers in passive irs-assisted systems with inexact oracles. *IEEE Access*, 13:154984–155002, 2025. doi: 10.1109/ACCESS.2025.3605249.

- Pozzobon, L., Ermis, B., Lewis, P., and Hooker, S. On the challenges of using black-box APIs for toxicity evaluation in research. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 7595–7609, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.472. URL <https://aclanthology.org/2023.emnlp-main.472/>.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. 2019.
- Saglam, B. and Kalogerias, D. Compatible gradient approximations for actor-critic algorithms, 2025. URL <https://arxiv.org/abs/2409.01477>.
- Schick, T., Udupa, S., and Schütze, H. Self-diagnosis and self-debiasing: A proposal for reducing corpus-based bias in nlp. *Transactions of the Association for Computational Linguistics*, 9:1408–1424, 12 2021. ISSN 2307-387X. doi: 10.1162/tac1_a_00434. URL https://doi.org/10.1162/tac1_a_00434.
- Team, G. Gemma 2: Improving open language models at a practical size, 2024. URL <https://arxiv.org/abs/2408.00118>.
- Thomas, K., Kelley, P. G., Tao, D., Meiklejohn, S., Vallis, O., Tan, S., Bratanić, B., Ferreira, F. T., Eranti, V. K., and Bursztein, E. Supporting Human Raters with the Detection of Harmful Content Using Large Language Models. In *2025 IEEE Symposium on Security and Privacy (SP)*, pp. 2772–2789, Los Alamitos, CA, USA, May 2025. IEEE Computer Society. doi: 10.1109/SP61157.2025.00082. URL <https://doi.ieeeecomputersociety.org/10.1109/SP61157.2025.00082>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is all you need. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Wang, B., Ping, W., Xiao, C., Xu, P., Patwary, M., Shoeybi, M., Li, B., Anandkumar, A., and Catanzaro, B. Exploring the limits of domain-adaptive training for detoxifying large-scale language models. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=v_0F4IZJZw.
- Weerawardhena, S., Kassianik, P., Nelson, B., Saglam, B., Vellore, A., Priyanshu, A., Vijay, S., Aufiero, M., Goldblatt, A., Burch, F., Li, E., He, J., Kedia, D., Oshiba, K., Yang, Z., Singer, Y., and Karbasi, A. Llama-3.1-foundationai-securityllm-8b-instruct technical report, 2025. URL <https://arxiv.org/abs/2508.01059>.
- Welbl, J., Glaese, A., Uesato, J., Dathathri, S., Mellor, J., Hendricks, L. A., Anderson, K., Kohli, P., Coppin, B., and Huang, P.-S. Challenges in detoxifying language models. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2021*, pp. 2447–2469, Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.findings-emnlp.210. URL <https://aclanthology.org/2021.findings-emnlp.210/>.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
- Xu, J., Ju, D., Li, M., Boureau, Y.-L., Weston, J., and Dinan, E. Recipes for safety in open-domain chatbots, 2021. URL <https://arxiv.org/abs/2010.07079>.
- Xu, Z., Jiang, F., Niu, L., Jia, J., Lin, B. Y., and Poovendran, R. SafeDecoding: Defending against jailbreak attacks via safety-aware decoding. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5587–5605, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.303. URL <https://aclanthology.org/2024.acl-long.303/>.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.

- Yang, K. and Klein, D. FUDGE: Controlled text generation with future discriminators. In Toutanova, K., Rumshisky, A., Zettlemoyer, L., Hakkani-Tur, D., Beltagy, I., Bethard, S., Cotterell, R., Chakraborty, T., and Zhou, Y. (eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 3511–3535, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.276. URL <https://aclanthology.org/2021.naacl-main.276/>.
- Zhan, H., Chen, C., Ding, T., Li, Z., and Sun, R. Unlocking black-box prompt tuning efficiency via zeroth-order optimization. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 14825–14838, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.871. URL <https://aclanthology.org/2024.findings-emnlp.871/>.
- Zhang, L., Li, B., Thekumparampil, K. K., Oh, S., and He, N. Dpzero: private fine-tuning of language models without backpropagation. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- Zhao, Y., Guo, Y., Weng, X., Tian, Y., Wang, W., and Chang, Y. Detoxifying large language models via the diversity of toxic samples. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 5869–5882, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.298. URL <https://aclanthology.org/2025.emnlp-main.298/>.
- Zheng, C., Ke, P., Zhang, Z., and Huang, M. Click: Controllable text generation with sequence likelihood contrastive learning. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 1022–1040, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.65. URL <https://aclanthology.org/2023.findings-acl.65/>.

A. Algorithm Pseudocode

Algorithm 1 Test-Time Iterative Detoxification via Embeddings (TIDE)

Require: Prompt embedding X_0 , model f , toxicity function h

```

1: for  $k = 0, 1, \dots, K$  do
2:   Sample  $\{U_i\}_{i=1}^N$ , each  $U_i \sim \mathcal{N}(0, I)$  and shaped  $(T, d)$  ▷ Tokenwise perturbations
3:   Compute zeroth-order gradient estimate:


$$g_k \leftarrow \frac{1}{N} \sum_{i=1}^N \frac{h(f(X_k + \mu U_i)) - h(f(X_k))}{\mu} U_i$$


4:   Normalize gradient direction:  $\hat{g}_k \leftarrow \frac{g_k}{\|g_k\|_2}$  ▷ Ensures controlled, scale-invariant update
5:   Update embeddings:  $X_{k+1} \leftarrow X_k - \eta \hat{g}_k$ 
6:   if  $\cos(X_{k+1}, X_0) < \kappa$  then ▷ Preserves embedding fidelity
7:      $X_{k+1} \leftarrow \Pi_{\mathcal{C}_\kappa}(X_{k+1}), \quad \mathcal{C}_\kappa = \{z : \cos(z, X_0) \geq \kappa\}$ 

8:   end if
9:   if  $h(f(X_{k+1})) < \tau$  then
10:    break ▷ Early stopping: embedding is sufficiently detoxified
11:   end if
12: end for
13: return  $X_{k+1}$ 
    
```

B. Experimental Details

B.1. Hyperparameter Selection

We first tuned hyperparameters on GPT-2 using the subset of the RTP dataset where the base model’s average toxicity exceeds 0.9, which corresponds to roughly 100 samples. We performed a grid search over $\mu = \{0.001, 0.005, 0.01, 0.05, 0.1, 0.5\}$, $N = \{4, 8, 16\}$, and $\eta = \{0.05, 0.1, 0.5, 1.0, 1.5, 2.0, 2.5\}$ at temperature 0.0. After selecting the best configuration, we constructed model-specific grids for the remaining models by scaling these values by $\frac{\bar{E}}{\sqrt{d}}$, where $\bar{E} = \frac{1}{|\mathcal{V}|} \sum_{i=1}^{|\mathcal{V}|} \|E_i\|_2$ is the mean norm of the rows of the embedding matrix $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ and $|\mathcal{V}|$ is the vocabulary size. We then ran this scaled, model-specific grid to select the best hyperparameters for each model. The final tuned values are listed in Table 3.

B.2. Implementation

We use vLLM (Kwon et al., 2023) to implement our method and to perform fast inference for base-model generations and fluency perplexity computation. vLLM supports efficient batch inference via continuous batching across prompts and decoding steps, enabling concurrent processing with shared model execution. In our setting, the $N + 1$ forward passes

Table 3. Model-specific tuned hyperparameters of TIDE, along with the embedding (or hidden) dimensionality and the larger model from the same family used to compute fluency perplexity. Our tuning procedure is described in Appendix B.1. For each model, we use the same set of hyperparameters across all benchmarks. The toxicity threshold $\tau = 0.5$ is used throughout.

Parameter	Llama 3.1-8B	Qwen3-4B	Gemma 2-2B	GPT-2 (774M)
Perturbation scale μ	0.03	0.01	0.05	0.1
# Monte Carlo samples	16	8	8	8
Stepsize η	0.3	0.65	1.0	1.5
Cosine similarity threshold κ	0.2	0.2	0.2	0.2
Dimensionality d	4096	2560	2304	1280
Perplexity model	Llama 3.1-70B	Qwen3-8B	Gemma 2-9B	GPT-2 XL (1.5B)

Table 4. Hyperparameter values evaluated in the sensitivity analysis. The tuned values, shared across all models, are shown in boldface. For a fair comparison with the original setting, we use $M = 3$ trials for all temperature values except 0.0.

Hyperparameter	Tested Values
Perturbation scale μ	$\{0.1\mu^*, 0.2\mu^*, 0.5\mu^*, 0.75\mu^*, \mu^*, 1.25\mu^*, 2.5\mu^*, 5\mu^*\}$
# Monte Carlo Samples	$\{1, 2, 4, \mathbf{8}, 16, 32, 64, 128\}$
Cosine similarity threshold κ	$\{0.0, \mathbf{0.2}, 0.4, 0.6, 0.8, 0.95\}$
Temperature	$\{0.0, \mathbf{0.1}, 0.3, 0.5, 0.7, 0.9, 1.0\}$

required per iteration are batched into a single call, making each iteration comparable in wall-clock time to a single forward pass. The total cost of the optimization loop is therefore approximately that of $\bar{K}$ sequential forward passes.

For SASA and RAD, we use the authors’ implementation,² which includes an optimized version of RAD that performs better in our setting than the original implementation.³ We use the latter repository to implement the remaining baselines—DExperts, DeStein, GeDi, and Toxification Reversal—which we run using Hugging Face’s transformers package (Wolf et al., 2020).

B.3. Sensitivity Analysis

We select Qwen3-4B and GPT-2 Large (774M), as their base-model responses yield the lowest and highest toxicity, respectively. The tested values and the resulting sensitivity analysis are shown in Table 4 and Figure 3, respectively. All other parameters are held at their tuned values in Table 3.

C. Supplementary Results

C.1. Perplexity Under a Shared Reference Model

Table 5. Same-family versus global (Qwen3-14B) perplexity for base-model continuations on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts). The same-family evaluator for each model is listed in Table 3.

Model	Same-family PPL	Global PPL (Qwen3-14B)
Llama 3.1-8B	3.54	4.46
Qwen3-4B	4.28	4.09
Gemma 2-2B	3.36	5.31
GPT-2 Large	4.13	5.07

A global, shared reference model for measuring fluency perplexity is an alternative but may introduce systematic cross-family bias. Model families differ in pretraining data and tokenization, so a sequence natural to one family may be unlikely under another—inflating perplexity independently of actual fluency. To verify, we analyze base-model outputs on RTP-challenging using Qwen3-14B (larger than every evaluated model) as a single global evaluator. Results are given in Table 5.

Perplexity increases by 23–58% for non-Qwen models while remaining largely unchanged for Qwen. This bias can invert expected orderings: under a single Qwen evaluator, the larger model (Llama) would appear less fluent—even though same-family evaluation confirms that Llama is indeed more fluent (3.54 vs. 4.28). Same-family evaluation avoids this artifact by measuring fluency relative to each model’s own distribution.

²<https://github.com/IBM/AISSteer360>

³<https://github.com/r-three/RAD>

C.2. Generalization Across Toxicity Measures

Table 6. Average maximum toxicity on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts), evaluated by Detoxify on the same continuations used in the main results. The RAD configuration shown is the one whose Perspective-API perplexity is closest to that of TIDE (Appendix C.5).

Model	TIDE (ours)	RAD
Llama 3.1-8B	0.123	0.127 ($\beta = 75$)
Qwen3-4B	0.091	0.237 ($\beta = 10$)
Gemma 2-2B	0.112	0.145 ($\beta = 50$)
GPT-2 Large	0.114	0.121 ($\beta = 50$)

By construction, TIDE is not constrained to any particular toxicity function—any continuous, real-valued scorer is compatible without modification. To verify, we let TIDE use the Perspective API during optimization as the oracle signal, but evaluate the optimized responses (along with the baselines) via Detoxify (Hanu & Unitary team, 2020), an open-source toxicity measure based on bidirectional models. Results are reported in Table 6.

At comparable perplexity, TIDE again achieves lower toxicity; RAD configurations that surpass TIDE come with substantially higher perplexity. Base-model toxicity under Detoxify is comparable to that under Perspective on these models, ruling out a “more lenient evaluator” artifact and confirming that the gains transfer across scorers.

C.3. Wall-Clock Time Comparison

Table 7. Average per-prompt wall-clock runtime on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts). Durations for TIDE include the full optimization loop with early stopping.

Model	TIDE (ours)			RAD ($\beta = 75$)
	Mean (s)	Median (s)	# Iterations $\bar{K}$	Mean (s)
Llama 3.1-8B	8.99	3.25	3.15	1.00
Qwen3-4B	2.93	1.96	1.63	1.16
Gemma 2-2B	4.54	2.25	2.40	1.17
GPT-2 Large	4.42	1.63	3.17	0.61

Per-prompt wall-clock runtimes, averaged over 1199 prompts, are provided in Table 7. TIDE is slower per input due to its iterative optimization. Given that it requires no preparation in advance (e.g., training additional models or collecting extra data), we consider this cost moderate, especially relative to the toxicity improvements it provides. The median is substantially lower than the mean: early stopping resolves most prompts quickly, with the mean inflated by a few high-toxicity inputs that require more iterations (see Table 7). On less challenging benchmarks (AttaQ and BOLD), $\bar{K}$ already drops to approximately 1 (see Appendix C.5).

C.4. High-Temperature Experiments

Table 8. High-temperature evaluation results for GPT-2 Large and Qwen3-4B on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts). Completions are generated with temperature 1.0 over 25 trials, and toxicity is measured using the Perspective API.

	Method	Average Max Toxicity	Perplexity	Toxic Rate	# Iterations $\bar{K}$
GPT-2 Large	Base model	0.912	7.66	0.991	–
	RAD ($\beta = 50$)	0.519	10.79	0.485	–
	RAD ($\beta = 75$)	0.401	11.30	0.337	–
	RAD ($\beta = 100$)	0.366	12.22	0.226	–
	SASA ($\beta = 50$)	0.501	11.53	0.576	–
	SASA ($\beta = 75$)	0.451	12.89	0.472	–
	SASA ($\beta = 100$)	0.448	13.57	0.443	–
	TIDE (ours)	0.324	13.98	0.000	2.22
	Base model	0.589	7.59	0.637	–
	RAD ($\beta = 50$)	0.577	6.47	0.617	–
Qwen3-4B	RAD ($\beta = 75$)	0.521	6.65	0.549	–
	RAD ($\beta = 100$)	0.481	7.06	0.492	–
	SASA ($\beta = 50$)	0.566	6.58	0.600	–
	SASA ($\beta = 75$)	0.566	6.62	0.596	–
	SASA ($\beta = 100$)	0.558	6.90	0.595	–
	TIDE (ours)	0.233	7.87	0.000	1.35

At high temperatures, variance-reduction techniques are needed to account for the increased stochasticity of the objective function caused by the more uniform token distribution that a high temperature admits. We consider a simple form of variance reduction: averaging over multiple completions. We replace each point evaluation $h(f(X))$ with the sample mean

$$\frac{1}{M'} \sum_{j=1}^{M'} h(f_j(X))$$

over M' independent completions (i.i.d. samples from the next-token distribution). This estimates the expected toxicity with respect to the generation stochasticity, $\mathbb{E}_f[h(f(X))]$, which is smooth as a function of X even when individual realizations are noisy—essentially pre-smoothing the objective before the finite-difference gradient estimation (2). The variance-reduction step is complementary to the optimization procedure of TIDE. The cost scales from $N + 1$ to $(N + 1) \cdot M'$ forward passes per iteration; $M' = 1$ recovers the standard TIDE setting at temperature 0.1. With batched inference via vLLM, all M' completions are parallelized, so wall-clock overhead again scales sublinearly.

We follow the experimental setting of Liu et al. (2021b), established primarily for stress-testing decoding-based methods that directly manipulate the sampling distribution. A temperature of 1.0 is used with $M = 25$ trials per sample. Experiments are conducted on GPT-2 Large and Qwen3-4B using the challenging subset of the RTP benchmark. We use $M' = 8$ averaged completions per evaluation of Φ and again $N = 8$ Monte Carlo samples. Results are provided in Table 8 alongside three configurations of RAD and SASA.

The results confirm that the proposed framework extends to high-temperature regimes when complemented with variance reduction, while operating entirely at the input level rather than modifying decoding as RAD and SASA do. The average number of iterations also drops considerably (from 1.63 to 1.35 on Qwen3-4B; from 3.17 to 2.22 on GPT-2) compared to the low-temperature setting. Averaging over many completions reduces the variance of the zeroth-order estimator and hence accelerates convergence, suggesting that a smaller M' may in fact suffice.

C.5. Complete Quantitative Results

Table 9. Evaluation results for Llama 3.1-8B on the AttaQ benchmark (1402 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations	$\bar{K}$
	Max	Mean				
Base model	0.222	0.166	3.11	0.101	–	
RAD ($\beta = 10$)	0.192	0.145	3.10	0.065	–	
RAD ($\beta = 50$)	0.146	0.107	3.35	0.030	–	
RAD ($\beta = 75$)	0.125	0.093	3.63	0.021	–	
RAD ($\beta = 100$)	0.113	0.085	3.92	0.019	–	
RAD ($\beta = 300$)	0.061	0.048	9.70	0.006	–	
RAD ($\beta = 500$)	0.045	0.037	19.88	0.000	–	
SASA ($\beta = 10$)	0.221	0.166	3.08	0.101	–	
SASA ($\beta = 50$)	0.212	0.161	3.09	0.093	–	
SASA ($\beta = 75$)	0.215	0.163	3.08	0.100	–	
SASA ($\beta = 100$)	0.212	0.160	3.10	0.092	–	
SASA ($\beta = 300$)	0.189	0.145	3.49	0.069	–	
SASA ($\beta = 500$)	0.169	0.131	5.17	0.060	–	
TIDE (ours)	0.138	0.112	3.20	0.001	1.14	

Table 10. Evaluation results for Qwen3-4B on the AttaQ benchmark (1402 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations	$\bar{K}$
	Max	Mean				
Base model	0.098	0.076	2.46	0.016	–	
RAD ($\beta = 10$)	0.091	0.072	2.46	0.011	–	
RAD ($\beta = 50$)	0.077	0.061	2.56	0.004	–	
RAD ($\beta = 75$)	0.073	0.058	2.65	0.002	–	
RAD ($\beta = 100$)	0.070	0.055	2.79	0.001	–	
RAD ($\beta = 300$)	0.053	0.041	5.78	0.002	–	
RAD ($\beta = 500$)	0.040	0.033	13.22	0.001	–	
SASA ($\beta = 10$)	0.103	0.078	2.44	0.016	–	
SASA ($\beta = 50$)	0.100	0.077	2.83	0.014	–	
SASA ($\beta = 75$)	0.100	0.077	2.84	0.014	–	
SASA ($\beta = 100$)	0.098	0.076	2.83	0.016	–	
SASA ($\beta = 300$)	0.097	0.076	2.61	0.011	–	
SASA ($\beta = 500$)	0.096	0.074	3.17	0.011	–	
TIDE (ours)	0.076	0.063	2.47	0.000	1.02	

Table 11. Evaluation results for Gemma 2-2B on the AttaQ benchmark (1402 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations	$\bar{K}$
	Max	Mean				
Base model	0.148	0.106	2.49	0.058	–	
RAD ($\beta = 10$)	0.130	0.091	2.50	0.037	–	
RAD ($\beta = 50$)	0.097	0.067	2.75	0.014	–	
RAD ($\beta = 75$)	0.084	0.059	3.01	0.007	–	
RAD ($\beta = 100$)	0.074	0.051	3.29	0.006	–	
RAD ($\beta = 300$)	0.047	0.035	6.75	0.002	–	
RAD ($\beta = 500$)	0.042	0.034	11.31	0.001	–	
SASA ($\beta = 10$)	0.155	0.109	2.45	0.062	–	
SASA ($\beta = 50$)	0.148	0.105	2.46	0.057	–	
SASA ($\beta = 75$)	0.144	0.102	2.47	0.052	–	
SASA ($\beta = 100$)	0.140	0.098	2.50	0.049	–	
SASA ($\beta = 300$)	0.111	0.075	3.67	0.026	–	
SASA ($\beta = 500$)	0.086	0.057	7.24	0.014	–	
TIDE (ours)	0.086	0.067	2.51	0.000	1.07	

Table 12. Evaluation results for GPT-2 Large (774M) on the AttaQ benchmark (1402 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations	$\bar{K}$
	Max	Mean				
Base model	0.179	0.134	3.36	0.072	–	
DExperts	0.165	0.125	3.38	0.057	–	
DeStein	0.055	0.037	5.77	0.008	–	
GeDi	0.155	0.116	3.54	0.045	–	
Toxification Reversal	0.110	0.075	4.74	0.028	–	
RAD ($\beta = 10$)	0.156	0.117	3.38	0.045	–	
RAD ($\beta = 50$)	0.117	0.088	3.51	0.015	–	
RAD ($\beta = 75$)	0.099	0.074	3.71	0.007	–	
RAD ($\beta = 100$)	0.089	0.066	3.98	0.005	–	
RAD ($\beta = 300$)	0.057	0.044	10.22	0.009	–	
RAD ($\beta = 500$)	0.045	0.039	18.20	0.000	–	
SASA ($\beta = 10$)	0.174	0.131	3.34	0.064	–	
SASA ($\beta = 50$)	0.174	0.132	3.35	0.065	–	
SASA ($\beta = 75$)	0.176	0.133	3.35	0.065	–	
SASA ($\beta = 100$)	0.174	0.132	3.35	0.063	–	
SASA ($\beta = 300$)	0.173	0.131	3.35	0.065	–	
SASA ($\beta = 500$)	0.173	0.130	3.37	0.062	–	
TIDE (ours)	0.108	0.088	3.44	0.000	1.11	

Table 13. Evaluation results for Llama 3.1-8B on the BOLD benchmark (23679 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations $\bar{K}$
	Max	Mean			
Base model	0.044	0.032	3.79	0.003	–
RAD ($\beta = 10$)	0.037	0.028	3.78	0.000	–
RAD ($\beta = 50$)	0.030	0.023	3.98	0.000	–
RAD ($\beta = 75$)	0.027	0.021	4.16	0.000	–
RAD ($\beta = 100$)	0.027	0.021	4.44	0.000	–
RAD ($\beta = 300$)	0.020	0.016	7.91	0.000	–
RAD ($\beta = 500$)	0.019	0.016	11.93	0.000	–
SASA ($\beta = 10$)	0.043	0.032	3.81	0.002	–
SASA ($\beta = 50$)	0.042	0.032	3.81	0.001	–
SASA ($\beta = 75$)	0.042	0.032	3.81	0.002	–
SASA ($\beta = 100$)	0.042	0.032	3.87	0.002	–
SASA ($\beta = 300$)	0.044	0.033	4.93	0.002	–
SASA ($\beta = 500$)	0.047	0.035	8.23	0.003	–
TIDE (ours)	0.032	0.026	3.77	0.000	1.00

Table 14. Evaluation results for Qwen3-4B on the BOLD benchmark (23679 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations $\bar{K}$
	Max	Mean			
Base model	0.031	0.024	3.35	0.000	–
RAD ($\beta = 10$)	0.030	0.024	3.33	0.000	–
RAD ($\beta = 50$)	0.026	0.021	3.39	0.000	–
RAD ($\beta = 75$)	0.025	0.020	3.50	0.000	–
RAD ($\beta = 100$)	0.024	0.020	3.64	0.000	–
RAD ($\beta = 300$)	0.022	0.018	6.24	0.000	–
RAD ($\beta = 500$)	0.023	0.020	11.58	0.000	–
SASA ($\beta = 10$)	0.030	0.024	3.36	0.000	–
SASA ($\beta = 50$)	0.031	0.025	3.37	0.000	–
SASA ($\beta = 75$)	0.032	0.025	3.37	0.000	–
SASA ($\beta = 100$)	0.032	0.025	3.37	0.000	–
SASA ($\beta = 300$)	0.032	0.025	3.82	0.000	–
SASA ($\beta = 500$)	0.033	0.026	4.93	0.000	–
TIDE (ours)	0.025	0.021	3.35	0.000	1.00

Table 15. Evaluation results for Gemma 2-2B on the BOLD benchmark (23679 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations	$\bar{K}$
	Max	Mean				
Base model	0.046	0.035	3.14	0.002	–	
RAD ($\beta = 10$)	0.040	0.030	3.15	0.000	–	
RAD ($\beta = 50$)	0.032	0.025	3.24	0.000	–	
RAD ($\beta = 75$)	0.030	0.024	3.38	0.001	–	
RAD ($\beta = 100$)	0.028	0.022	3.59	0.000	–	
RAD ($\beta = 300$)	0.025	0.020	6.54	0.000	–	
RAD ($\beta = 500$)	0.026	0.022	10.40	0.000	–	
SASA ($\beta = 10$)	0.046	0.035	3.17	0.002	–	
SASA ($\beta = 50$)	0.045	0.035	3.20	0.002	–	
SASA ($\beta = 75$)	0.045	0.034	3.24	0.002	–	
SASA ($\beta = 100$)	0.045	0.034	3.31	0.003	–	
SASA ($\beta = 300$)	0.041	0.030	5.22	0.002	–	
SASA ($\beta = 500$)	0.038	0.028	9.53	0.000	–	
TIDE (ours)	0.035	0.029	3.14	0.000	1.00	

Table 16. Evaluation results for GPT-2 Large (774M) on the BOLD benchmark (23679 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations	$\bar{K}$
	Max	Mean				
Base model	0.059	0.042	4.80	0.014	–	
DExperts	0.052	0.038	4.90	0.008	–	
DeStein	0.020	0.016	7.22	0.000	–	
GeDi	0.051	0.036	4.89	0.006	–	
Toxification Reversal	0.040	0.028	7.62	0.002	–	
RAD ($\beta = 10$)	0.047	0.034	4.75	0.003	–	
RAD ($\beta = 50$)	0.035	0.026	4.83	0.001	–	
RAD ($\beta = 75$)	0.033	0.025	4.94	0.000	–	
RAD ($\beta = 100$)	0.031	0.023	5.17	0.000	–	
RAD ($\beta = 300$)	0.033	0.029	7.85	0.000	–	
RAD ($\beta = 500$)	0.035	0.032	9.39	0.000	–	
SASA ($\beta = 10$)	0.060	0.042	4.81	0.011	–	
SASA ($\beta = 50$)	0.058	0.041	4.80	0.011	–	
SASA ($\beta = 75$)	0.059	0.042	4.80	0.012	–	
SASA ($\beta = 100$)	0.059	0.041	4.81	0.013	–	
SASA ($\beta = 300$)	0.058	0.041	4.81	0.012	–	
SASA ($\beta = 500$)	0.058	0.041	4.81	0.011	–	
TIDE (ours)	0.038	0.030	4.77	0.000	1.02	

Table 17. Evaluation results for Llama 3.1-8B on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations $\bar{K}$
	Max	Mean			
Base model	0.572	0.475	3.54	0.617	–
RAD ($\beta = 10$)	0.425	0.341	3.65	0.404	–
RAD ($\beta = 50$)	0.240	0.186	4.47	0.141	–
RAD ($\beta = 75$)	0.194	0.150	5.19	0.098	–
RAD ($\beta = 100$)	0.167	0.129	6.07	0.069	–
RAD ($\beta = 300$)	0.098	0.078	17.03	0.022	–
RAD ($\beta = 500$)	0.078	0.066	30.02	0.010	–
SASA ($\beta = 10$)	0.570	0.475	3.51	0.615	–
SASA ($\beta = 50$)	0.579	0.479	3.53	0.634	–
SASA ($\beta = 75$)	0.575	0.476	4.27	0.631	–
SASA ($\beta = 100$)	0.576	0.478	3.57	0.629	–
SASA ($\beta = 300$)	0.581	0.489	4.53	0.624	–
SASA ($\beta = 500$)	0.546	0.462	6.42	0.572	–
TIDE (ours)	0.184	0.164	5.21	0.060	3.15

Table 18. Evaluation results for Qwen3-4B on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations $\bar{K}$
	Max	Mean			
Base model	0.341	0.278	4.28	0.295	–
RAD ($\beta = 10$)	0.275	0.220	4.41	0.202	–
RAD ($\beta = 50$)	0.174	0.134	5.18	0.088	–
RAD ($\beta = 75$)	0.146	0.112	5.33	0.067	–
RAD ($\beta = 100$)	0.129	0.099	7.06	0.054	–
RAD ($\beta = 300$)	0.084	0.068	22.40	0.026	–
RAD ($\beta = 500$)	0.072	0.061	47.23	0.023	–
SASA ($\beta = 10$)	0.342	0.277	4.28	0.304	–
SASA ($\beta = 50$)	0.331	0.270	4.26	0.287	–
SASA ($\beta = 75$)	0.333	0.270	3.82	0.290	–
SASA ($\beta = 100$)	0.331	0.267	4.30	0.289	–
SASA ($\beta = 300$)	0.310	0.247	4.67	0.261	–
SASA ($\beta = 500$)	0.281	0.226	5.67	0.215	–
TIDE (ours)	0.144	0.122	4.20	0.003	1.63

Table 19. Evaluation results for Gemma 2-2B on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations $\bar{K}$
	Max	Mean			
Base model	0.547	0.458	3.36	0.588	–
RAD ($\beta = 10$)	0.380	0.303	3.53	0.353	–
RAD ($\beta = 50$)	0.206	0.161	4.31	0.121	–
RAD ($\beta = 75$)	0.167	0.129	5.01	0.085	–
RAD ($\beta = 100$)	0.149	0.115	5.99	0.069	–
RAD ($\beta = 300$)	0.100	0.083	17.99	0.029	–
RAD ($\beta = 500$)	0.089	0.075	30.30	0.019	–
SASA ($\beta = 10$)	0.556	0.458	3.39	0.598	–
SASA ($\beta = 50$)	0.545	0.448	3.39	0.589	–
SASA ($\beta = 75$)	0.533	0.438	3.41	0.575	–
SASA ($\beta = 100$)	0.522	0.428	3.47	0.561	–
SASA ($\beta = 300$)	0.397	0.311	6.17	0.415	–
SASA ($\beta = 500$)	0.283	0.214	11.77	0.252	–
TIDE (ours)	0.151	0.113	4.57	0.002	2.40

Table 20. Evaluation results for GPT-2 Large (774M) on the “challenging” subset of the RealToxicityPrompts benchmark (1199 prompts). Completions are generated with temperature 0.1 over three trials, and toxicity is measured using the Perspective API.

Method	Average Toxicity		Perplexity	Toxic Rate	# Iterations $\bar{K}$
	Max	Mean			
Base model	0.591	0.495	4.13	0.639	–
DExperts	0.527	0.460	4.18	0.562	–
DeStein	0.116	0.087	8.72	0.062	–
GeDi	0.297	0.245	7.46	0.250	–
Toxification Reversal	0.334	0.271	6.63	0.321	–
RAD ($\beta = 10$)	0.419	0.336	4.39	0.401	–
RAD ($\beta = 50$)	0.209	0.157	5.50	0.120	–
RAD ($\beta = 75$)	0.159	0.122	6.25	0.066	–
RAD ($\beta = 100$)	0.134	0.102	7.21	0.046	–
RAD ($\beta = 300$)	0.070	0.058	19.46	0.006	–
RAD ($\beta = 500$)	0.062	0.055	35.79	0.005	–
SASA ($\beta = 10$)	0.592	0.492	4.20	0.638	–
SASA ($\beta = 50$)	0.580	0.482	4.18	0.624	–
SASA ($\beta = 75$)	0.587	0.488	4.20	0.630	–
SASA ($\beta = 100$)	0.579	0.481	4.20	0.621	–
SASA ($\beta = 300$)	0.570	0.468	4.18	0.610	–
SASA ($\beta = 500$)	0.557	0.455	4.17	0.596	–
TIDE (ours)	0.156	0.122	5.53	0.003	3.17